

Lecture Notes on Agentic AI

Jawwad Ahmed Shamsi
FAST-NUCES, Pakistan

August 26, 2026

3 How Machines Represent and Understand Language

This lecture takes a top-down pass through six terms you will hear constantly for the rest of the course. The goal here is conceptual fluency, not architectural depth – we return to the full mathematical structure of the Transformer in a later lecture. Learning objectives:

1. Define a Large Language Model (LLM) in terms of what it is actually trained to do
2. Explain tokenization and why it is not simply “splitting on spaces”
3. Explain how words become vectors, and why nearby vectors mean related meaning
4. Explain attention as a mechanism for context-dependent meaning
5. Explain self-supervised learning and why it made LLMs possible at scale
6. Describe the Transformer at a high level as the architecture that ties all of the above together

3.1 Large Language Model (LLM)

Definition

A **Large Language Model** is a neural network trained to **predict the next token** in a sequence of text, given everything that came before it.

This definition is deceptively simple, and worth sitting with. Everything an LLM appears to do – answering questions, writing code, holding a conversation – emerges from repeatedly applying one operation: given some text so far, predict what token comes next.

Question

If it only predicts the next word, how can it hold a whole conversation? Once it predicts one token, that token is appended to the input, and the model predicts the *next* token given the new, longer input. Repeating this one operation thousands of times, one token at a time, is sufficient to produce entire essays, conversations, and programs.

3.2 Tokenization

Before a model can predict “the next token,” text must be broken into tokens in the first place.

Definition

Tokenization is the process of breaking input text into discrete units called **tokens**. Tokens are not simply words – tokenization also captures meaningful sub-word units, such as prefixes and suffixes.

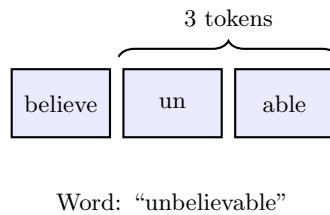


Figure 1: Tokenization of “unbelievable” into sub-word tokens. The suffix “un” and root “believe” are captured as separate, meaningful units rather than the whole word being treated as one indivisible block.

Example

Why not just split on spaces? Splitting on spaces would treat “run,” “running,” and “runs” as three completely unrelated tokens, giving the model no signal that they share a root meaning. Sub-word tokenization captures the shared root (**run**) plus the suffix (**ing**, **s**), which helps the model generalize across related word forms it may never have seen exactly before.

3.3 Vectors and Vectorization

Tokens are symbols. A neural network needs numbers.

Definition

Vectorization maps each token to a point (a vector) in an n -dimensional space, such that tokens with similar meaning are positioned close together.

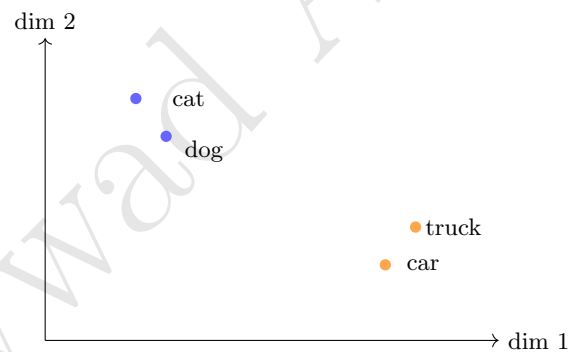


Figure 2: A toy 2-dimensional slice of vector space. Animals cluster together; vehicles cluster together, far from the animals. Real models use hundreds or thousands of dimensions, not two.

Question

Is this the same as the word2vec embeddings from last semester? Same underlying idea – meaning as position in space – but word2vec produces a *fixed* vector per word, regardless of context. The vectors an LLM uses are *contextual*: the same word can shift position in the space depending on what surrounds it. That shift is exactly what attention (next section) computes.

3.4 Attention

Definition

Attention is a mechanism that lets a model derive **contextual meaning** by allowing each token's vector to be adjusted based on other, relevant tokens nearby.

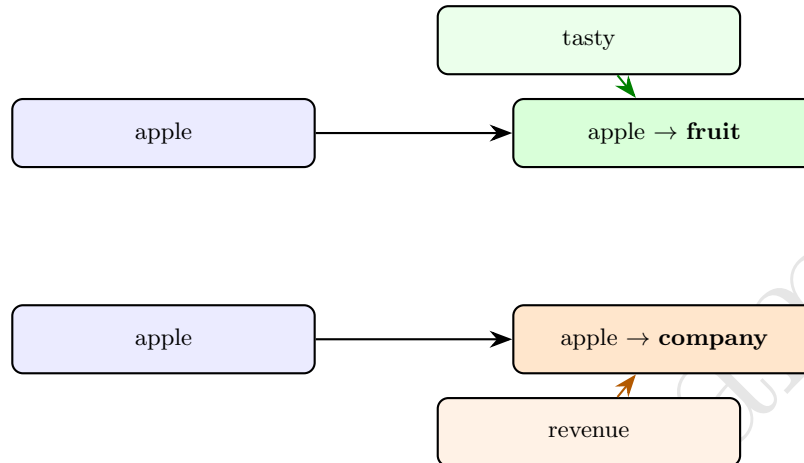


Figure 3: Attention shifts the vector for “apple” toward different regions of meaning depending on which words are nearby – “tasty” pulls it toward *fruit*; “revenue” pulls it toward *company*.

Example

Pronoun resolution. Consider: “The trophy did not fit in the suitcase because *it* was too big.” Attention is what allows the model to connect “it” back to “trophy” rather than “suitcase” – by attending strongly to “trophy” when processing the word “it,” the model folds that context into “it”’s vector representation.

3.5 Self-Supervised Learning

A natural question: where do the correct answers come from, to train a model that predicts vectors and next tokens correctly, across the entire internet of text?

Definition

Self-supervised learning is a training method where the **data’s own structure** provides the labels – for example, by hiding a word in a sentence and training the model to predict it. No human needs to manually label the data.

Example

Fill in the blank, at internet scale. Given the sentence “The cat sat on the _____,” the true next word (“mat,” say) is already present in the original text – it does not need a human annotator to supply it. Applying this at the scale of the entire internet is what made training LLMs on trillions of tokens economically feasible: there is no manual labeling bottleneck.

3.6 The Transformer

We now have all the pieces: tokens, vectors, attention, and a training method that scales. The Transformer is the architecture that assembles them.

Definition

The **Transformer** is a neural network architecture built from **stacked layers of attention blocks and feedforward networks**, which repeatedly refine token vectors until the model can predict the next token accurately.

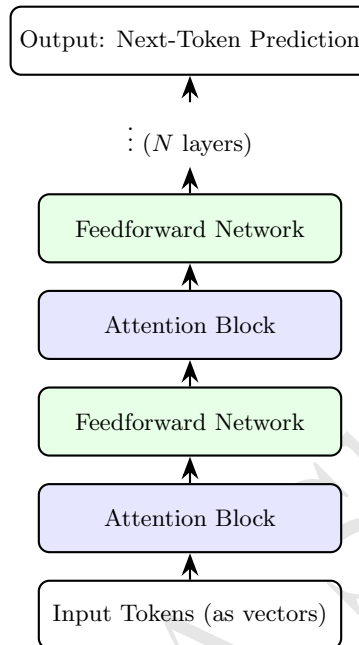


Figure 4: A Transformer, at a glance: input vectors pass through repeated layers of attention and feedforward computation before producing a next-token prediction. Full internal detail of each block is covered in a later lecture.

Question

Is “Transformer” the same thing as “LLM”? The two terms are often used interchangeably, but they are not identical. The **Transformer** is the algorithm/architecture – the engine. An **LLM** is the trained product built using that engine (trained on a specific dataset, at a specific scale). Not every Transformer is necessarily large enough to be called an LLM, and in principle other architectures could be used to build a language model.

3.7 Summary

We took a top-down pass through six foundational terms: an LLM is a model trained to predict the next token; tokenization breaks text into meaningful sub-word units; vectorization places tokens in a space where meaning corresponds to position; attention adjusts those positions based on context; self-supervised learning provides labels for free from the data itself, enabling training at internet scale; and the Transformer is the stacked architecture that ties all of this together. A later lecture returns to the Transformer in full mathematical detail.

Exercises

1. **Next-token intuition.** Given the input “The capital of France is,” what token would you expect an LLM to predict next, and why? What does this tell you about what the model has “learned” from its training data?
2. **Tokenization practice.** Break the word “unhappiness” into what you believe are its most meaningful sub-word tokens, and explain your reasoning.

3. **Vector space.** Using the toy example in Figure 2, where would you expect the word “motorcycle” to be positioned relative to “car” and “dog”? Why?
4. **Attention, applied.** Write one sentence containing a word whose meaning depends entirely on context (like “apple”). Identify the two different meanings and the context word that would push the model toward each one.
5. **Self-supervised vs. supervised.** Explain, in your own words, why self-supervised learning was necessary to train models at the scale of the modern internet, and what would have to change if we relied only on human-labeled data instead.

Jawwad A Shamsi